

1.RAD Model (Rapid Application Development)

The **RAD Model** is a linear sequential software development process that emphasizes **short development cycles** using **component-based construction** and **prototyping**. It is suitable when requirements are well understood and the project scope is limited, allowing the system to be developed quickly.

RAD focuses on faster and higher-quality development through:

- User requirement gathering using workshops
 - Prototyping with early user feedback
 - Reuse of software components
 - Use of automated tools
 - Less formal communication
-

Phases of RAD Model

1. **Business Modelling** – Defines information flow between business functions.
 2. **Data Modelling** – Converts business data into data entities and relationships.
 3. **Process Modelling** – Transforms data objects to support business processes.
 4. **Application Generation** – Uses automated tools to build software quickly.
 5. **Testing & Turnover** – Reused components reduce testing time; new parts are tested.
-

Advantages

- Fast development
 - Flexible to changes
 - Reusability of components
-

Disadvantages

- Requires skilled developers
- Not suitable for high-risk or very small projects
- Needs continuous user involvement

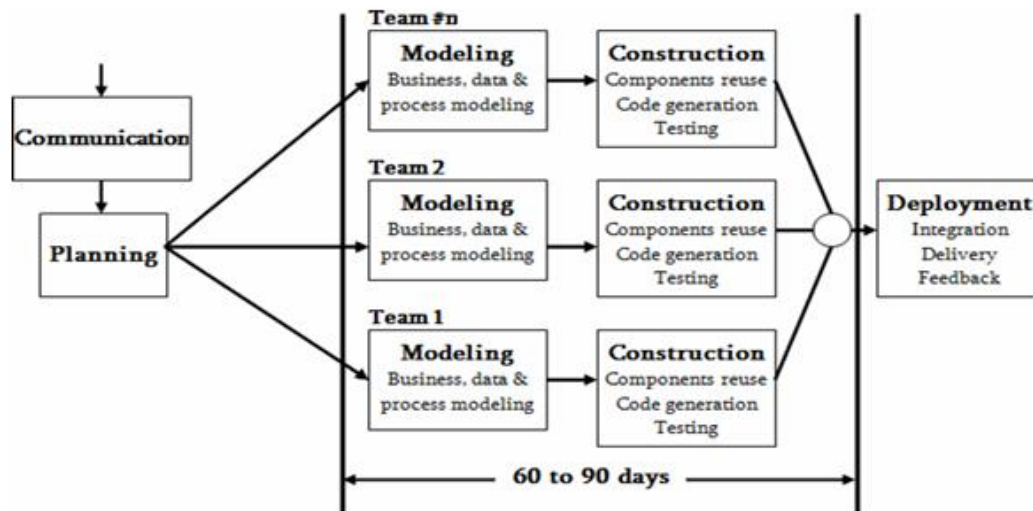


Figure : Flowchart of RAD model

2. Agile Model

The **Agile Model** is a software development approach based on **iterative and incremental development**. The word *Agile* means swift and flexible. In this model, the project is divided into small iterations that usually last **1 to 4 weeks**, and each iteration goes through planning, design, coding, and testing to deliver a working product.

Agile reduces project risk and development time by breaking the system into smaller parts and allowing frequent changes based on customer feedback.

Phases of Agile Model

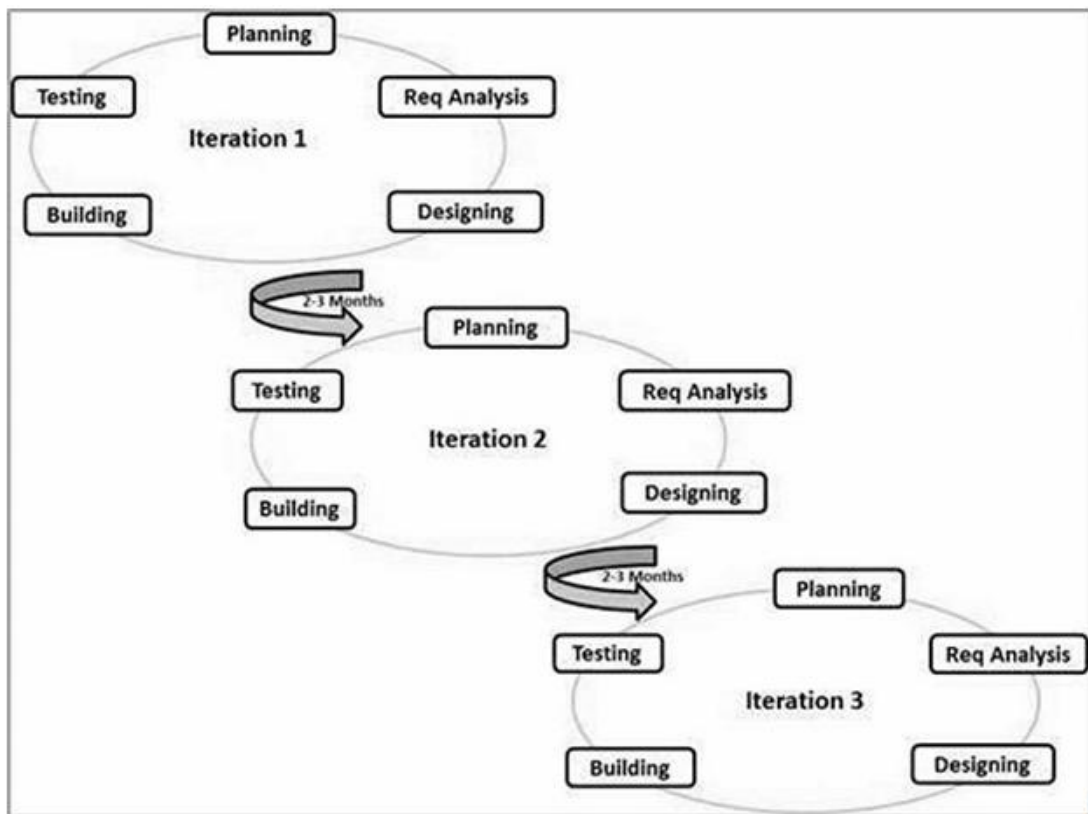
1. **Requirements Gathering** – Identify business needs and feasibility.
 2. **Design** – Prepare system design using diagrams.
 3. **Construction/Iteration** – Develop and improve working product.
 4. **Testing** – Check product for bugs and performance.
 5. **Deployment** – Release product to users.
 6. **Feedback** – Collect user feedback for improvement.
-

☀ Advantages

- Frequent delivery of working software
 - Accepts changes easily
 - Good customer communication
 - Reduces development time
-

⚠ Disadvantages

- Lack of proper documentation
- Maintenance becomes difficult after project completion



3. Emergence of Software Engineering

Software Engineering evolved gradually as computer technology advanced and software systems became more complex. To overcome the limitations of informal programming methods, structured and systematic approaches were developed.

Important Stages in Emergence

1. Early Computer Programming

Programs were small and written in assembly language using the **build-and-fix** approach without proper planning.

2. High-Level Language Programming

High-level languages like **FORTRAN, ALGOL, and COBOL** reduced effort and allowed larger programs, but informal methods were still used.

3. Control Flow-based Design

Programmers began designing control structures using **flowcharts** and **structured programming** techniques.

4. Data Structure-oriented Design

Focus shifted to designing important **data structures first**, followed by program logic.

5. Data Flow-oriented Design

Emphasized identifying data items and their processing using **Data Flow Diagrams (DFDs)**.

6. Object-Oriented Design

Introduced **object-oriented techniques** to improve code reuse and easier maintenance.

Exploratory Style of Software Development

The **Exploratory Style of Software Development** is an informal **build-and-fix** approach in which programmers rely on intuition rather than systematic methods. In this style, developers begin coding without proper planning and continuously fix errors until the software works correctly.

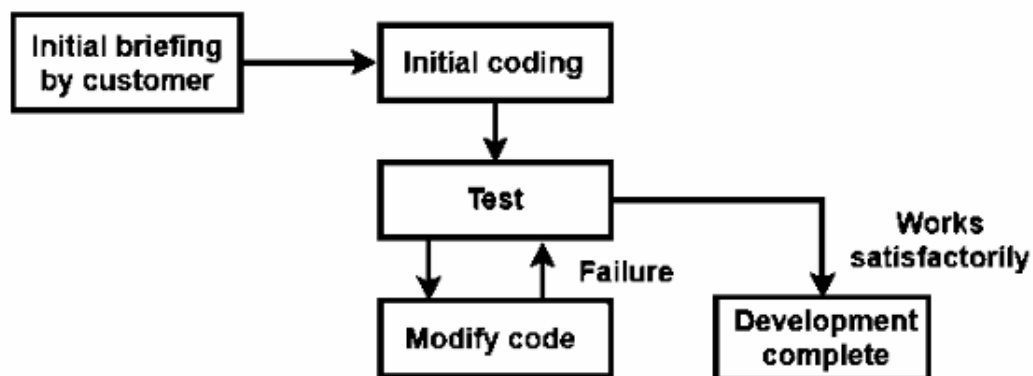
This model does not follow any defined rules or structured steps for development.

Development Process

The process starts with a **customer briefing**, followed by initial coding by the programmer.

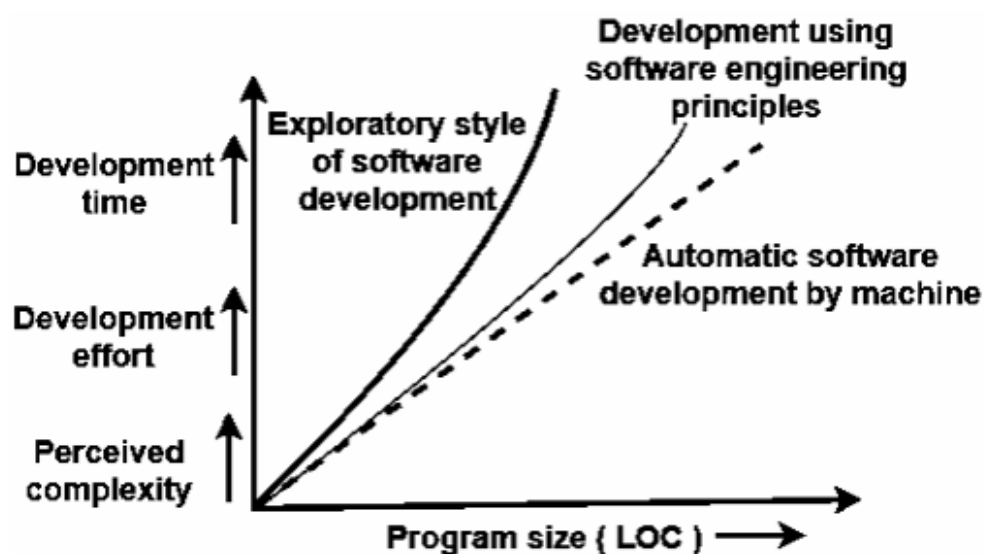
The program is then tested, bugs are fixed, and the cycle of coding and testing continues until a satisfactory product is achieved.

This style is mostly used by students for laboratory assignments and is not commonly used in industries.



Problems with Exploratory Style

As the size of the program increases, the **development effort, time, and cost increase exponentially**. This makes the model unsuitable for large and complex software systems. Human limitations in handling complexity cause the difficulty to grow rapidly.



✖ Shortcomings

- Development becomes expensive and time-consuming for large projects
- Produces unstructured and poor-quality code
- Software becomes difficult to maintain
- Not suitable for team-based development